

DAA LAB – 7 REPORT

Chepoori Sai Vivek (CH.SC.U4CSE24108)

Q) Job Sequencing

To implement the Greedy algorithm for the Job Sequencing with Deadlines problem and determine the optimal sequence of jobs that maximizes total profit.

The profits of the jobs are:

{22, 19, 29, 28, 30, 21, 27, 25, 24, 26, 14, 27, 19, 11}

The deadlines of the jobs are:

{3, 3, 8, 6, 7, 5, 10, 4, 6, 12, 13, 2, 14, 1}

Code :

```
#include <stdio.h>

#define MAX 20

struct Job {
    int id;
    int profit;
    int deadline;
};

void sortJobs(struct Job jobs[], int n) {
    int i, j;
    struct Job temp;

    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(jobs[j].profit < jobs[j+1].profit) {
                temp = jobs[j];
                jobs[j] = jobs[j+1];
                jobs[j+1] = temp;
            }
        }
    }
}

int main() {
    int n = 14;
    struct Job jobs[MAX] = {
        {1,22,3},{2,19,3},{3,29,8},{4,28,6},{5,30,7},
        {6,21,5},{7,27,10},{8,25,4},{9,24,6},{10,26,12},
        {11,14,13},{12,27,2},{13,19,14},{14,11,1}
    };

    int slots[MAX];
    int i, j;
    int maxProfit = 0;

    for(i = 0; i < MAX; i++)
        slots[i] = -1;

    sortJobs(jobs, n);

    for(i = 0; i < n; i++) {
```

```

int main() {
    int n = 14;
    struct Job jobs[MAX] = {
        {1,22,3},{2,19,3},{3,29,8},{4,28,6},{5,30,7},
        {6,21,5},{7,27,10},{8,25,4},{9,24,6},{10,26,12},
        {11,14,13},{12,27,2},{13,19,14},{14,11,1}
    };

    int slots[MAX];
    int i, j;
    int maxProfit = 0;

    for(i = 0; i < MAX; i++)
        slots[i] = -1;

    sortJobs(jobs, n);

    for(i = 0; i < n; i++) {
        for(j = jobs[i].deadline; j > 0; j--) {
            if(slots[j] == -1) {
                slots[j] = i;
                maxProfit += jobs[i].profit;
                break;
            }
        }
    }

    printf("Selected Jobs:\n");
    for(i = 1; i <= 14; i++) {
        if(slots[i] != -1)
            printf("J%d ", jobs[slots[i]].id);
    }

    printf("\nTotal Profit = %d\n", maxProfit);

    return 0;
}

```

OUTPUT :

```

CH.SC.U4CSE24108> cd /mnt/c/Users/user/Documents/Haskell
CH.SC.U4CSE24108> gcc daa.c -o daa
CH.SC.U4CSE24108> ./daa
Selected Jobs:
J6 J12 J1 J8 J9 J4 J5 J3 J7 J10 J11 J13
Total Profit = 292
CH.SC.U4CSE24108> |

```

Time Complexity

Let n be the number of jobs.

1. Sorting Phase (Bubble Sort)

Two nested loops are used to sort jobs in decreasing order of profit.

- Outer loop runs $(n - 1)$ times.
- Inner loop runs up to $(n - 1)$ times.

In the worst case, total comparisons are proportional to: $n \times n = n^2$

Therefore, **Time complexity of sorting** = $O(n^2)$

Justification: Bubble sort compares each element with others using nested loops.

2. Scheduling Phase

Again, two nested loops are used:

- Outer loop runs n times (for each job).
- Inner loop may run up to n times (checking slots backward).

In the worst case, for every job all previous slots are checked.

Thus, $n \times n = n^2$

Therefore, **Time complexity of scheduling** = $O(n^2)$

Overall Time Complexity

Sorting: $O(n^2)$ Scheduling: $O(n^2)$

Total: $O(n^2) + O(n^2) = O(n^2)$

Final Answer: **Time Complexity** = $O(n^2)$

Space Complexity

The program uses:

- `profit[n]`, `deadline[n]`, `job[n]`, `slot[n]`

All arrays are of size proportional to n .

No recursion and no additional dynamic memory is used.

Therefore,

Space Complexity = $O(n)$